

MATERI PEMROGRAMAN C++

Perulangan (Looping)

Mata Pelajaran	Pemrograman C++
Topik	Perulangan (for, while, do-while)
Tingkat	Pemula - Menengah
Bahasa	Indonesia

1. Pendahuluan

Perulangan (loop) adalah salah satu struktur kontrol fundamental dalam pemrograman. Dengan perulangan, kita dapat menjalankan blok kode yang sama secara berulang tanpa harus menuliskannya berkali-kali. Dalam bahasa C++, terdapat tiga jenis perulangan utama: **for**, **while**, dan **do-while**.

Perulangan sangat berguna untuk memproses data dalam jumlah besar, menelusuri elemen array, menampilkan pola tertentu, serta mengulang operasi hingga suatu kondisi terpenuhi.

Konsep Dasar Perulangan

- **Inisialisasi** — menentukan nilai awal variabel kontrol.
- **Kondisi** — ekspresi yang dievaluasi setiap iterasi; loop berjalan selama bernilai *true*.
- **Increment/Decrement** — memperbarui variabel kontrol setiap iterasi.
- **Body** — blok kode yang dijalankan pada setiap iterasi.

2. Perulangan for

Perulangan **for** digunakan ketika jumlah iterasi sudah diketahui sebelumnya. Sintaks perulangan for menggabungkan inisialisasi, kondisi, dan increment dalam satu baris.

Sintaks

```
for (inisialisasi; kondisi; increment/decrement) {  
    // blok kode yang diulang  
}
```

Contoh 1 — Menampilkan Angka 1 s.d. 10

```
#include <iostream>  
using namespace std;  
  
int main() {  
    for (int i = 1; i <= 10; i++) {  
        cout << i << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output: 1 2 3 4 5 6 7 8 9 10

Contoh 2 — Menghitung Jumlah Bilangan Genap

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int jumlah = 0;  
    for (int i = 2; i <= 20; i += 2) {
```

```

        jumlah += i;
    }
    cout << "Jumlah bilangan genap 2-20: " << jumlah << endl;
    return 0;
}

```

Output: Jumlah bilangan genap 2-20: 110

Contoh 3 — Perulangan Menurun (Countdown)

```

for (int i = 10; i >= 1; i--) {
    cout << i << " ";
}
// Output: 10 9 8 7 6 5 4 3 2 1

```

3. Perulangan while

Perulangan **while** digunakan ketika jumlah iterasi tidak diketahui sebelumnya dan bergantung pada suatu kondisi. Kondisi diperiksa *sebelum* blok kode dijalankan.

Sintaks

```

while (kondisi) {
    // blok kode yang diulang
    // pastikan ada perubahan kondisi agar tidak infinite loop
}

```

Contoh 1 — Input Validasi

```

#include <iostream>
using namespace std;

int main() {
    int angka;
    cout << "Masukkan angka positif: ";
    cin >> angka;

    while (angka <= 0) {
        cout << "Angka harus positif! Coba lagi: ";
        cin >> angka;
    }
    cout << "Angka valid: " << angka << endl;
    return 0;
}

```

Contoh 2 — Menghitung Digit Angka

```

#include <iostream>
using namespace std;

int main() {

```

```

int n = 123456, count = 0;
while (n != 0) {
    n /= 10;
    count++;
}
cout << "Jumlah digit: " << count << endl;
return 0;
}

```

Output: Jumlah digit: 6

** Perhatian: Pastikan kondisi while selalu akan menjadi false pada suatu titik, agar tidak terjadi infinite loop.*

4. Perulangan do-while

Perulangan **do-while** mirip dengan **while**, namun kondisi diperiksa *setelah* blok kode dijalankan. Artinya, blok kode **selalu dijalankan minimal satu kali** meskipun kondisi bernilai false sejak awal.

Sintaks

```

do {
    // blok kode yang diulang
} while (kondisi); // <-- perhatikan titik koma di sini

```

Contoh 1 — Menu Pilihan

```

#include <iostream>
using namespace std;

int main() {
    int pilihan;
    do {
        cout << "=== MENU ===" << endl;
        cout << "1. Tambah Data" << endl;
        cout << "2. Hapus Data" << endl;
        cout << "0. Keluar" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;
    } while (pilihan != 0);

    cout << "Program selesai." << endl;
    return 0;
}

```

Contoh 2 — Menghitung Faktorial

```

#include <iostream>
using namespace std;

int main() {
    int n = 5, hasil = 1, i = 1;

```

```

do {
    hasil *= i;
    i++;
} while (i <= n);
cout << "5! = " << hasil << endl;
return 0;
}

```

Output: 5! = 120

5. Perulangan Bersarang (Nested Loop)

Perulangan bersarang adalah perulangan di dalam perulangan. Setiap kali perulangan luar berjalan satu iterasi, perulangan dalam menyelesaikan seluruh iterasinya. Nested loop sering digunakan untuk memproses data dua dimensi seperti matriks atau mencetak pola.

Contoh 1 — Mencetak Pola Bintang

```

#include <iostream>
using namespace std;

int main() {
    int baris = 5;
    for (int i = 1; i <= baris; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}

```

Output:

```

*
* *
* * *
* * * *
* * * * *

```

Contoh 2 — Perkalian (Tabel Kali)

```

for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i*j << "\t";
    }
    cout << endl;
}

```

Output:

```

1   2   3
2   4   6

```

3 6 9

6. Kata Kunci break dan continue

break — Menghentikan Perulangan

Kata kunci **break** digunakan untuk menghentikan perulangan secara paksa sebelum kondisi akhir tercapai.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 6) break;    // hentikan saat i = 6  
    cout << i << " ";  
}  
// Output: 1 2 3 4 5
```

continue — Melewati Iterasi

Kata kunci **continue** digunakan untuk melewati sisa blok kode pada iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya.

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue; // lewati bilangan genap  
    cout << i << " ";  
}  
// Output: 1 3 5 7 9
```

7. Perbandingan Ketiga Jenis Perulangan

Aspek	for	while	do-while
Pengecekan kondisi	Sebelum iterasi	Sebelum iterasi	Setelah iterasi
Eksekusi minimal	0 kali (jika kondisi false)	0 kali (jika kondisi false)	Selalu 1 kali
Penggunaan terbaik	Jumlah iterasi diketahui	Jumlah iterasi tidak diketahui	Harus jalan minimal sekali (misal menu)
Inisialisasi	Di dalam header	Di luar loop	Di luar loop

8. Latihan Soal

Soal 1 (Mudah): Tampilkan bilangan ganjil dari 1 sampai 50 menggunakan perulangan for.

Soal 2 (Mudah): Hitung jumlah semua angka dari 1 sampai N, di mana N diinput oleh pengguna.

Soal 3 (Menengah): Buat program yang menampilkan tabel perkalian dari 1x1 hingga 10x10 menggunakan nested loop.

Soal 4 (Menengah): Buat program kalkulator sederhana dengan menu (tambah, kurang, kali, bagi) menggunakan do-while.

Soal 5 (Tantangan): Buat program untuk mencari bilangan prima dari 2 hingga 100. Gunakan nested loop dan break/continue.

Selamat belajar! Pemahaman yang baik tentang perulangan adalah fondasi penting untuk menjadi programmer C++ yang handal.